

05506026 Digital Image Processing (Java)

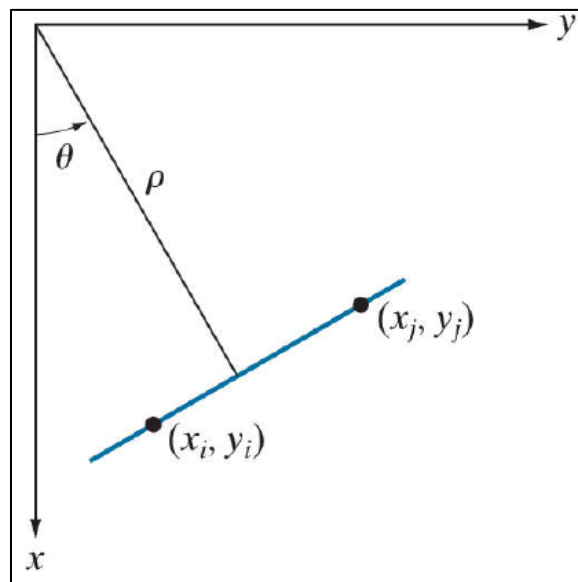
Lab 10

Objectives

1. Understand how to perform global Hough transform

Global Processing using Hough Transform

Hough Transform is an approach to accept and eliminate edgels based on predefined global properties of regions of interest. The process will transform image intensities in xy -space into $\rho\theta$ -plane as we call hough array.



Hough Transform has 4 steps which are

1. Find each edge pixel in a binary image
2. Apply the line equation to those edgels and update hough array ($\rho\theta$ -plane) using the values

3. Apply threshold to hough array to find the values with high frequency which most likely to form a line
4. Draw lines with those values

First, we create a method to perform Hough Transform with one threshold parameter as percentile of line pixels.

```
ImageManager.java
class ImageManager
{
    ...
    public void houghTransform(double percent)
    {
    }
    ...
}
```

So, Step 1 and 2, after we apply Canny Edge Detector, we will find those white pixels of edge pixels, apply the line equation, and update hough array.

ImageManager.java

```

class ImageManager
{
    ...
    public void houghTransform(double percent)
    {
        //The image should be converted to edge map first

        //Work out how the hough space is quantized
        int numOfTheta = 720;
        double thetaStep = Math.PI / numOfTheta;
        int highestR = (int)(Math.max(width, height) * Math.sqrt(2));
        int centreX = width / 2;
        int centreY = height / 2;
        System.out.println("Hough array w: "+numOfTheta+" height: "+2*highestR);

        //Create the hough array and initialize to zero
        int[][] houghArray = new int[numOfTheta][2*highestR];
        for(int i = 0; i < numOfTheta; i++)
        {
            for(int j = 0; j < 2*highestR; j++)
            {
                houghArray[i][j] = 0;
            }
        }

        //Step 1 - find each edge pixel
        //Find edge points and vote in array
        for(int y = 0; y < height; y++)
        {
            for(int x = 0; x < width; x++)
            {
                int pointColor = img.getRGB(x, y) & 0xff;
                if (pointColor != 0)
                {
                    //Edge pixel found
                    for (int i = 0; i < numOfTheta; i++)
                    {
                        //Step 2 - Apply the line equation and update hough array
                        //Work out the r values for each theta step
                        int r = (int)((x - centreX) * Math.cos(i * thetaStep) +
                                    (y - centreY) * Math.sin(i * thetaStep));
                    }
                }
            }
        }
    }
}

```

```

        //Move all values into positive range for display purposes
        r = r + highestR;
        if (r < 0 || r >= 2 * highestR) continue;

        //Increment hough array
        houghArray[i][r]++;
    }
}
}
...
}

```

Step 3 and 4, after hough array is updated, we apply a threshold value with the percentage of the max value which implies the longest line in the image.

ImageManager.java

```

class ImageManager
{
    ...
    public void houghTransform(double percent)
    {
        //Step 1 - find each edge pixel
        //Step 2 - Apply the line equation and update hough array
        ...
        //Step 3 - Apply threshold to hough array to find line

        int maxHough = 0;

        for(int i = 0; i < numOfTheta; i++)
        {
            for(int j = 0; j < 2 * highestR; j++)
            {
                //Find the max hough value for the thresholding operation
                if(houghArray[i][j] > maxHough)
                {
                    maxHough = houghArray[i][j];
                }
            }
        }

        //Set the threshold limit
        int threshold = (int)(percent * maxHough);
    }
}

```

```

//Step 4 - Draw lines
// Search for local peaks above threshold to draw

for(int i = 0; i < numOfTheta; i++)
{
    for(int j = 0; j < 2 * highestR; j++)
    {
        // only consider points above threshold
        if(houghArray[i][j] >= threshold)
        {
            // see if local maxima
            boolean draw = true;
            int peak = houghArray[i][j];
            for(int k = -1; k <= 1; k++)
            {
                for(int l = -1; l <= 1; l++)
                {
                    //not seeing itself
                    if (k == 0 && l == 0) continue;
                    int testTheta = i + k;
                    int testOffset = j + l;

                    if (testOffset < 0 || testOffset >= 2*highestR) continue;
                    if (testTheta < 0) testTheta = testTheta + numOfTheta;
                    if (testTheta >= numOfTheta) testTheta = testTheta - numOfTheta;

                    if (houghArray[testTheta][testOffset] > peak)
                    {
                        //found bigger point
                        draw = false;
                        break;
                    }
                }
            }
            //point found is not local maxima
            if (!draw) continue;

            //if local maxima, draw red back
            double tsin = Math.sin(i*thetaStep);
            double tcos = Math.cos(i*thetaStep);

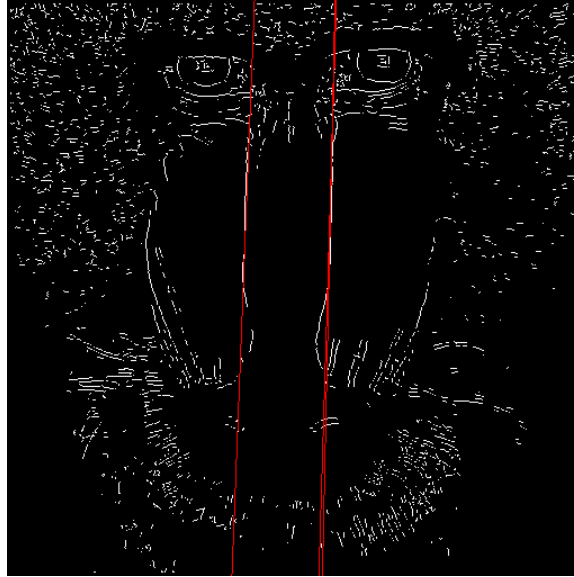
```

```

        if (i <= numOfTheta / 4 || i >= (3 * numOfTheta) / 4)
        {
            for(int y = 0; y < height; y++)
            {
                //vertical line
                int x = (int) (((j - highestR) - ((y - centreY) * tsin)) / tcos) + centreX;
                if(x < width && x >= 0)
                {
                    int redColor = (255 << 16) | (0 << 8) | 0;
                    img.setRGB(x, y, redColor);
                }
            }
        }
        else
        {
            for(int x = 0; x < width; x++)
            {
                //horizontal line
                int y = (int) (((j - highestR) - ((x - centreX) * tcos)) / tsin) + centreY;
                if(y < height && y >= 0)
                {
                    int redColor = (255 << 16) | (0 << 8) | 0;
                    img.setRGB(x, y, redColor);
                }
            }
        }
    }
}
...
}

```

This will produce an image as follow, which is the result of Hough transform using 0.8 as the percentage threshold.



Quest 014

Implement the method of Hough Transform and perform percent as 0.5

The player who finishes this task will be awarded **20 EXP** and **1 Move**.

Deadline: 7 September 2025